

CAJERO AUTOMÁTICO BÁSICO

Análisis, algoritmo, pseudocódigo, flujograma, prueba de escritorio y codificación

ENUNCIADO DEL EJERCICIO

Construir un cajero automático con las opciones: consultar saldo, depositar, retirar, mostrar movimientos y salir. El usuario comenzará con un saldo de \$100 y el programa permanecerá activo hasta seleccionar Salir. No se permitirán depósitos negativos, retiros negativos, retiros mayores que el saldo disponible ni opciones inexistentes. Al finalizar se mostrarán los depósitos y retiros realizados, el total depositado, el total retirado y el saldo final. Se utilizarán do-while, switch, contadores y acumuladores.

1. ANÁLISIS DE DATOS

1.1 Entrada de datos

- Opción del menú: entero del 1 al 5.
- Monto de depósito: dato real mayor que cero.
- Monto de retiro: dato real mayor que cero y menor o igual que el saldo disponible.

1.2 Proceso

- Inicializar el saldo en \$100, los dos contadores en cero y los dos acumuladores en cero.
- Mostrar el menú al menos una vez mediante do-while y evaluar la opción con switch.
- Consultar y mostrar el saldo actual.
- Validar que un depósito sea positivo; si es válido, sumarlo al saldo, incrementar su contador y acumular su total.
- Validar que un retiro sea positivo y no exceda el saldo; si es válido, restarlo, incrementar su contador y acumular su total.
- Mostrar los movimientos acumulados cuando se seleccione la opción 4.
- Rechazar opciones diferentes de 1 a 5.
- Repetir el menú hasta seleccionar 5 y luego mostrar el resumen final.

1.3 Salida de datos

- Saldo actual.
- Mensajes de operación aceptada o rechazada.
- Cantidad de depósitos y retiros realizados.
- Total depositado y total retirado.
- Saldo final.
- Mensaje de opción inexistente.

2. ALGORITMO

1. **Inicio.**
2. Declarar las variables de opción, saldo, monto, contadores y acumuladores.
3. Asignar \$100 al saldo e inicializar contadores y acumuladores en cero.

4. Mostrar el menú y leer la opción.
5. Si la opción es 1, mostrar el saldo.
6. Si es 2, leer el depósito y aceptarlo únicamente si es positivo; actualizar saldo, contador y total depositado.
7. Si es 3, leer el retiro y comprobar que sea positivo y no exceda el saldo; actualizar saldo, contador y total retirado.
8. Si es 4, mostrar los movimientos y valores acumulados.
9. Si es 5, mostrar el mensaje de finalización.
10. Si la opción no existe, mostrar un error.
11. Repetir el menú mientras la opción sea diferente de 5.
12. Mostrar el resumen final completo.
13. **Fin.**

3. PSEUDOCÓDIGO

Algoritmo CajeroAutomaticoConMovimientos

Definir opcion, depositosRealizados, retirosRealizados Como Entero;

Definir saldo, monto, totalDepositado, totalRetirado Como Real;

saldo <- 100;

depositosRealizados <- 0;

retirosRealizados <- 0;

totalDepositado <- 0;

totalRetirado <- 0;

Repetir

 Escribir "1. Consultar saldo";

 Escribir "2. Depositar";

 Escribir "3. Retirar";

 Escribir "4. Mostrar movimientos";

 Escribir "5. Salir";

 Leer opcion;

 Segun opcion Hacer

 1:

 Escribir "Saldo actual: \$", saldo;

 2:

 Escribir "Monto a depositar:";

 Leer monto;

 Si monto > 0 Entonces

 saldo <- saldo + monto;

 depositosRealizados <- depositosRealizados + 1;

 totalDepositado <- totalDepositado + monto;

 SiNo

 Escribir "Error: el depósito debe ser positivo";

 FinSi;

 3:

 Escribir "Monto a retirar:";

 Leer monto;

 Si monto <= 0 Entonces

 Escribir "Error: el retiro debe ser positivo";

 SiNo

 Si monto > saldo Entonces

 Escribir "Error: fondos insuficientes";

 SiNo

 saldo <- saldo - monto;

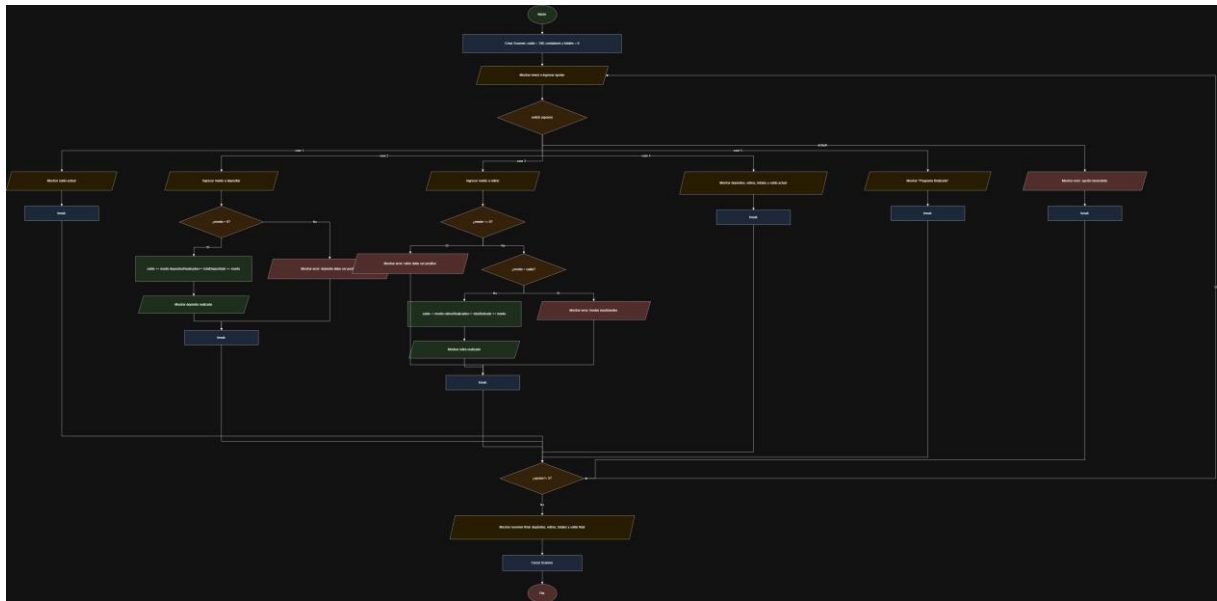
 retirosRealizados <- retirosRealizados + 1;

 totalRetirado <- totalRetirado + monto;

 FinSi;

```
    FinSi;
4:
    Escribir "Depósitos realizados: ", depositosRealizados;
    Escribir "Retiros realizados: ", retirosRealizados;
    Escribir "Total depositado: $", totalDepositado;
    Escribir "Total retirado: $", totalRetirado;
    Escribir "Saldo actual: $", saldo;
5:
    Escribir "Programa finalizado";
De Otro Modo:
    Escribir "Error: opción inexistente";
FinSegun;
Hasta Que opcion = 5;
Escribir "Depósitos realizados: ", depositosRealizados;
Escribir "Retiros realizados: ", retirosRealizados;
Escribir "Total depositado: $", totalDepositado;
Escribir "Total retirado: $", totalRetirado;
Escribir "Saldo final: $", saldo;
FinAlgoritmo
```

4. DIAGRAMA DE FLUJO



5. PRUEBA DE ESCRITORIO

Caso 1: Depósito y retiro válidos

Variable	Valor
saldo inicial	\$100,00
depósito	\$50,00
retiro	\$30,00
depósitos realizados	1
retiros realizados	1
total depositado	\$50,00
total retirado	\$30,00
saldo final	\$120,00

Operaciones:

Después del depósito: saldo = $100 + 50 = \$150,00$.

Después del retiro: saldo = $150 - 30 = \$120,00$.

Los dos movimientos son válidos y actualizan sus contadores y acumuladores.

Caso 2: Montos negativos rechazados

Variable	Valor
saldo inicial	\$100,00
depósito intentado	-\$20,00
retiro intentado	-\$10,00
depósitos realizados	0
retiros realizados	0
total depositado	\$0,00
total retirado	\$0,00
saldo final	\$100,00

Operaciones:

$-20 \leq 0$: el depósito se rechaza y no modifica ningún valor.

$-10 \leq 0$: el retiro se rechaza y no modifica ningún valor.

Saldo final = \$100,00.

Caso 3: Fondos insuficientes y opción inexistente

Variable	Valor
saldo inicial	\$100,00
opción intentada	9 (inexistente)
retiro intentado	\$150,00
depósito válido	\$25,00
depósitos realizados	1

Variable	Valor
retiros realizados	0
total depositado	\$25,00
total retirado	\$0,00
saldo final	\$125,00

Operaciones:

La opción 9 se rechaza y el menú se muestra nuevamente.

\$150,00 > \$100,00: el retiro se rechaza por fondos insuficientes.

Depósito válido: saldo = 100 + 25 = \$125,00.

Resumen de los casos

Caso	Depósitos	Retiros	Total depositado	Total retirado	Saldo final
1	1	1	\$50,00	\$30,00	\$120,00
2	0	0	\$0,00	\$0,00	\$100,00
3	1	0	\$25,00	\$0,00	\$125,00

6. CÓDIGO

JAVA

```
import java.util.Scanner;

public class CajeroMovimientos {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        int opcion, depositosRealizados = 0, retirosRealizados = 0;
        double saldo = 100, monto;
        double totalDepositado = 0, totalRetirado = 0;

        do {
            System.out.println("\n===== CAJERO AUTOMÁTICO =====");
            System.out.println("1. Consultar saldo");
            System.out.println("2. Depositar");
            System.out.println("3. Retirar");
            System.out.println("4. Mostrar movimientos");
            System.out.println("5. Salir");
            System.out.print("Seleccione una opción: ");
            opcion = entrada.nextInt();

            switch (opcion) {
                case 1:
                    System.out.printf("Saldo actual: $%.2f\n", saldo);
                    break;
                case 2:
                    System.out.print("Monto a depositar: $");
                    monto = entrada.nextDouble();
                    if (monto > 0) {
                        saldo += monto;
                        depositosRealizados++;
                        totalDepositado += monto;
                        System.out.println("Depósito realizado correctamente.");
                    } else {
                        System.out.println("Error: el depósito debe ser positivo.");
                    }
                    break;
                case 3:
                    System.out.print("Monto a retirar: $");
                    monto = entrada.nextDouble();
                    if (monto <= 0) {
                        System.out.println("Error: el retiro debe ser positivo.");
                    } else if (monto > saldo) {
                        System.out.println("Error: fondos insuficientes.");
                    } else {
                        saldo -= monto;
                        retirosRealizados++;
                        totalRetirado += monto;
                        System.out.println("Retiro realizado correctamente.");
                    }
                    break;
                case 4:
                    System.out.println("Depósitos realizados: " + depositosRealizados);
                    System.out.println("Retiros realizados: " + retirosRealizados);
                    System.out.printf("Total depositado: $%.2f\n", totalDepositado);
                    System.out.printf("Total retirado: $%.2f\n", totalRetirado);
                    System.out.printf("Saldo actual: $%.2f\n", saldo);
                    break;
                case 5:
                    System.out.println("Programa finalizado.");
                    break;
                default:
                    System.out.println("Error: opción inexistente.");
            }
        } while (opcion != 5);
    }
}
```

```
System.out.println("\n===== RESUMEN FINAL =====");
System.out.println("Depósitos realizados: " + depositosRealizados);
System.out.println("Retiros realizados: " + retirosRealizados);
System.out.printf("Total depositado: $%.2f%n", totalDepositado);
System.out.printf("Total retirado: $%.2f%n", totalRetirado);
System.out.printf("Saldo final: $%.2f%n", saldo);
entrada.close();
}
}
```

C++

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    int opcion, depositosRealizados = 0, retirosRealizados = 0;
    double saldo = 100, monto;
    double totalDepositado = 0, totalRetirado = 0;

    do {
        cout << "\n===== CAJERO AUTOMATICO =====\n";
        cout << "1. Consultar saldo\n2. Depositar\n3. Retirar\n";
        cout << "4. Mostrar movimientos\n5. Salir\n";
        cout << "Seleccione una opcion: ";
        cin >> opcion;

        switch (opcion) {
            case 1:
                cout << fixed << setprecision(2) << "Saldo actual: $" << saldo << "\n";
                break;
            case 2:
                cout << "Monto a depositar: $";
                cin >> monto;
                if (monto > 0) {
                    saldo += monto;
                    depositosRealizados++;
                    totalDepositado += monto;
                    cout << "Deposito realizado correctamente.\n";
                } else {
                    cout << "Error: el deposito debe ser positivo.\n";
                }
                break;
            case 3:
                cout << "Monto a retirar: $";
                cin >> monto;
                if (monto <= 0) {
                    cout << "Error: el retiro debe ser positivo.\n";
                } else if (monto > saldo) {
                    cout << "Error: fondos insuficientes.\n";
                } else {
                    saldo -= monto;
                    retirosRealizados++;
                    totalRetirado += monto;
                    cout << "Retiro realizado correctamente.\n";
                }
                break;
            case 4:
                cout << "Depositos realizados: " << depositosRealizados << "\n";
                cout << "Retiros realizados: " << retirosRealizados << "\n";
                cout << fixed << setprecision(2);
                cout << "Total depositado: $" << totalDepositado << "\n";
                cout << "Total retirado: $" << totalRetirado << "\n";
                cout << "Saldo actual: $" << saldo << "\n";
                break;
            case 5:
                cout << "Programa finalizado.\n";
                break;
            default:
                cout << "Error: opcion inexistente.\n";
        }
    } while (opcion != 5);

    cout << "\n===== RESUMEN FINAL =====\n";
    cout << "Depositos realizados: " << depositosRealizados << "\n";
    cout << "Retiros realizados: " << retirosRealizados << "\n";
```

```
cout << fixed << setprecision(2);  
cout << "Total depositado: $" << totalDepositado << "\n";  
cout << "Total retirado: $" << totalRetirado << "\n";  
cout << "Saldo final: $" << saldo << "\n";  
return 0;  
}
```